

Abstrakte Rechnermodelle

Ziele

- Suche nach Berechnungsmodellen (“abstrakte Computer”)
- Zeige, dass viele Berechnungsmodelle äquivalent sind
- Wähle ein möglichst einfaches Berechnungsmodell

Alphabete und Wörter

- Was ist ein Problem? – Für bestimmte Eingaben sollen bestimmte Ausgaben produziert werden.
- Ein- und Ausgaben sind **Wörter** über einem **Alphabet** Σ , einer endlichen Menge von Symbolen
 - Praktisch: Strings über $\Sigma = \text{UTF-8}$ oder $\Sigma = \{0,1\}$
- Oft: $\Sigma = \{0,1\}$.
- Σ^k : die Menge aller Wörter der **Länge** k , z.B. $\{0,1\}^2 = \{00,01,10,11\}$
- Das **leere Wort** ϵ , also das Wort der Länge 0, d.h. $\Sigma^0 = \{\epsilon\}$
- Der **Kleensche Abschluss** $\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k$ von Σ enthält alle **Wörter** über Σ
- Man kann Σ^* abzählen: $\Sigma^* = \{\epsilon, 0,1,00,01,10,11,000, \dots\}$

Probleme als Relation

- Im Allgemeinen entspricht ein Problem einer Relation $R \subseteq \Sigma^* \times \Sigma^*$
- Ein Paar (x, y) liegt in R , wenn y eine zulässige Ausgabe zur Eingabe x ist.
- Bsp: Primfaktorbestimmung
 - Zu einer natürlichen Zahl $q \geq 2$ suchen wir einen Primfaktor. Kodiere Zahlen binär: $i \in \mathbb{N} \rightarrow \text{bin}(i) \in \{0,1\}^*$.
 - Die entsprechende Relation ist

$$R = \{(x, y) \in \{0,1\}^* \times \{0,1\}^* \mid x = \text{bin}(q), y = \text{bin}(p), p, q \in \mathbb{N}, q \geq 2, p \text{ prim}, p|q\}$$

Problem als Funktion

- Diese Relation ist eine Funktion $f: \Sigma^* \rightarrow \Sigma^*$, wenn Ergebnis *eindeutig*. Dann ist $f(x)$ die eindeutige Ausgabe zum Problem x .
- Beispiel:
 - Zu zwei natürlichen Zahlen $i, j \in \mathbb{N}$ suchen wir das Produkt.
 - Zum Trennen der Zahlen i, j erweitern wir das Alphabet um ein Trennsymbol $\#$, d.h. $\Sigma = \{0, 1, \#\}$

$$f: \Sigma^* \rightarrow \Sigma^*: \text{bin}(i)\#\text{bin}(j) \mapsto \text{bin}(i \cdot j)$$

Eingabecheck

- Oft ist nicht jedes Wort aus Σ^* eine gültige Eingabe.
- Bsp: $f: \Sigma^* \rightarrow \Sigma^*: bin(i) \# bin(j) \mapsto bin(i \cdot j)$
- Bei ungültigen Eingaben wird oft ein Fehler ausgegeben
- Bsp:

$$f: \Sigma^* \rightarrow \Sigma^*: x \mapsto \begin{cases} bin(i \cdot j), & \text{falls } x = bin(i) \# bin(j) \\ \#, & \text{sonst} \end{cases}$$

- Dieser Syntaxcheck gehört zu jedem Algorithmus.
- Einen Fehler durch $\#$ zu kodieren ist nur auf dieser Folie unsere Konvention.
- Hier keine Typtheorie oder Monaden (wie Exceptions), für “schöne” Fehler.

Entscheidungsproblem als Sprache

- Viele Probleme lassen sich als Ja-Nein-Fragen formulieren.
- Derartige Entscheidungsprobleme sind von der Form $f: \Sigma^* \rightarrow \{0,1\}$ mit Interpretation 0=„Nein“ und 1=„Ja“.
- Sei $L = f^{-1}(1) \subseteq \Sigma^*$ die Menge der Eingaben mit Antwort „Ja“. L ist eine Teilmenge der Wörter über dem Alphabet Σ . Eine solche Teilmenge wird als **Sprache** bezeichnet.
- Beispiel: Graphzusammenhang.
 - Für einen gegebenen Graphen G soll bestimmt werden, ob G zusammenhängend ist.
 - Der Graph G liegt in Kodierung $code(G) \in \Sigma^*$ vor, z.B. binär kodierte Adjazenzmatrix.
 - $L = \{w \in \Sigma^* | \exists Graph\ G: w = code(G) \wedge G\ zusammenhängend\}$

Zentrale Fragestellung

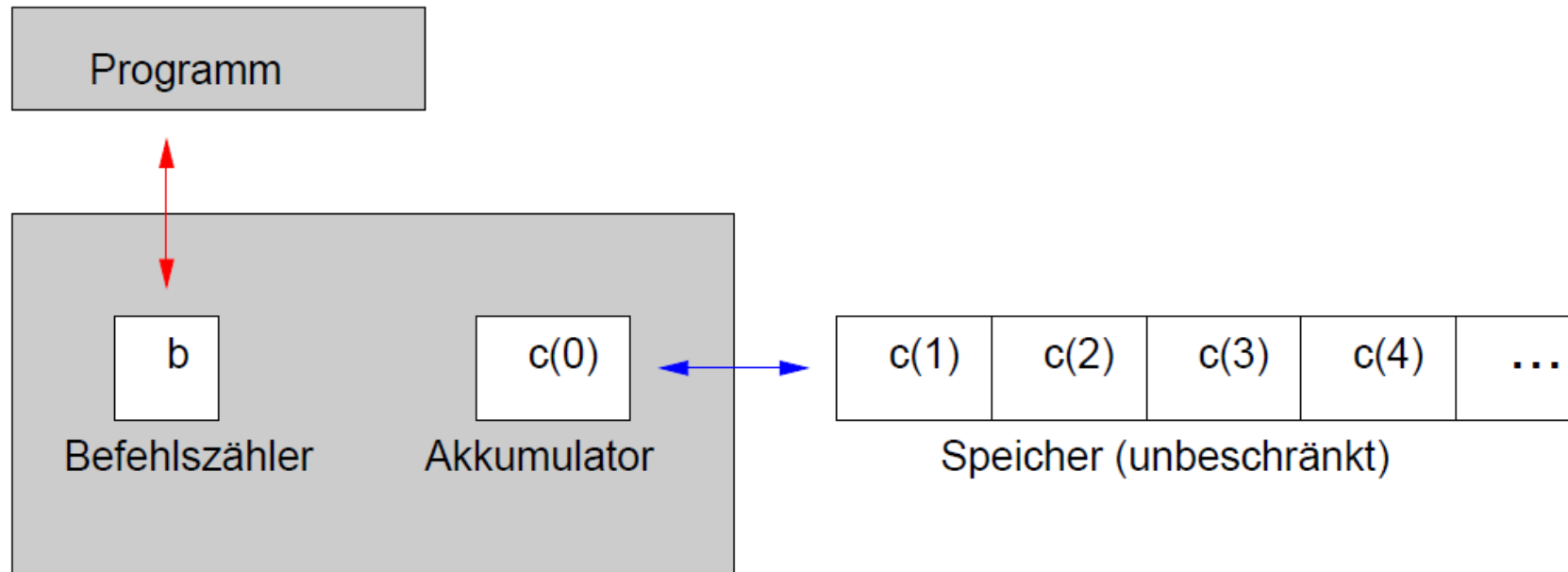
- Welche Funktionen sind durch einen Computer berechenbar?
- Welche Sprachen kann ein Computer entscheiden?
- Um diese Fragen in einem mathematisch exakten Sinne klären zu können, benötigen wir neben den formalen Beschreibungen von Problemen, die wir schon entwickelt haben, noch ein mathematisches Computer- bzw. Maschinenmodell.
- Zur Vereinfachung betrachten wir meistens Entscheidungsprobleme
 - Wir werden sehen: keine große Einschränkung

Registermaschine (RAM)

Computer und Abstraktion

- Nachteile von Intel/AMD/ARM/NVIDIA/etc. Computern für uns:
 - Kein einheitliches Modell
 - Viel zu kompliziert
- Wir benötigen eine Abstraktion
- Grundsätzlich: elementare Datenstrukturen sind oft einfach gewählt, man kann dann damit komplexere Datenstrukturen modellieren.
 - Ganze Zahlen aus natürlichen Zahlen
 - Rationale Zahlen als Paar einer ganzen und einer natürlichen Zahl
 - Strings als natürliche Zahl

Registermaschine (genauer: Random Access Machine; RAM)



Befehlsatz:

LOAD, STORE, ADD, SUB, DIV

INDLOAD, INDSTORE, INDADD, INDSUB, INDDIV

CLOAD, CADD, CSUB, CDIV

GOTO, IF c(0)?x THEN GOTO j (wobei ? aus {=, <, <=, >, >=}), END

Erläuterung ausgewählter RAM-Befehle

- LOAD i : $c(0) := c(i), b := b + 1;$
- STORE i : $c(i) := c(0), b := b + 1;$
- ADD i : $c(0) := c(0) + c(i), b := b + 1;$
- INDLOAD i : $c(0) := c(c(i)), b := b + 1;$
- CLOAD i : $c(0) := i, b := b + 1;$
- GOTO j : $b := j$
- IF $c(0) = x$ GOTO j : $b := j$ falls $c(0) = x$, sonst $b := b + 1;$
- Andere Befehle und weitere Details siehe Wegener Buch.

Funktionsweise der Registermaschine

- Der Speicher der RAM ist unbeschränkt und besteht aus den Registern $c(0)$, $c(1)$, $c(2)$, $c(3)$, $\dots$
- Die Inhalte der Register sind natürliche Zahlen, die beliebig groß sein können.
- Die Eingabe sind ebenfalls natürliche Zahlen, initial in den ersten Registern abgespeichert.
- Der Befehlszähler startet mit dem Wert 1. Ausgeführt wird jeweils der Befehl in derjenigen Zeile auf den der Befehlszähler verweist.
- Die Rechnung stoppt sobald der Befehl END erreicht ist.
- Die Ausgabe befindet sich nach dem Stoppen ebenfalls in den ersten Registern.

Diskussion der Registermaschine

- Auf der Registermaschine können wir offensichtlich alle Befehle wie beispielsweise Schleifen und Rekursionen, die wir von Programmiersprachen gewohnt sind, realisieren.
- Für die Bemessung der Rechenzeit gibt es unterschiedliche Modelle, z.B.
 - Uniformes Kostenmaß: Jeder Schritt zählt eine Zeiteinheit.
 - Logarithmisches Kostenmaß: Die Laufzeitkosten eines Schrittes sind proportional zur binären Länge der Zahlen in den angesprochenen Registern.
- Aus Sicht eines Programmierers mag die Registermaschine recht elementar aussehen. Aus Sicht eines mathematischen Modells ist sie relativ komplex.
- Um das Argumentieren zu erleichtern, betrachten wir ein noch elementareres Maschinenmodell, das 1936 von Alan Turing eingeführt wurde.

Deterministische Turingmaschine

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO
THE ENTSCHIEDUNGSPROBLEM

By A. M. TURING.

[Received 28 May, 1936.—Read 12 November, 1936.]

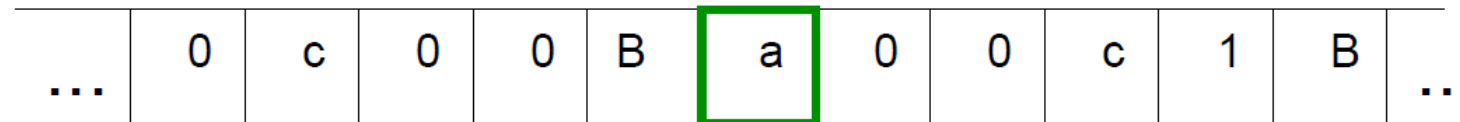
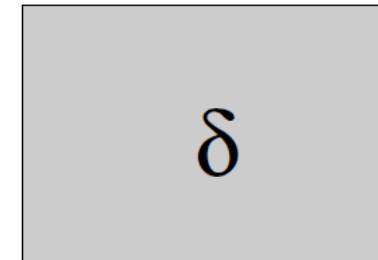
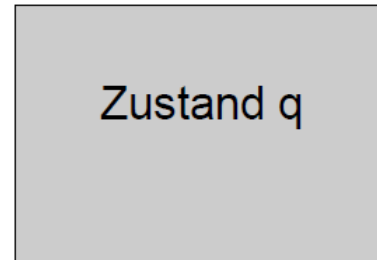
The “computable” numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are, however, the same in each case, and I have chosen the computable numbers for explicit treatment as involving the least cumbrous technique. I hope shortly to give an account of the relations of the computable numbers, functions, and so forth to one another. This will include a development of the theory of functions of a real variable expressed in terms of computable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

*Proceedings of the London
Mathematical Society,
Volume s2-42, Issue 1, 1937,
Pages 230–265*

Deterministische Turingmaschinen (TM bzw. DTM)



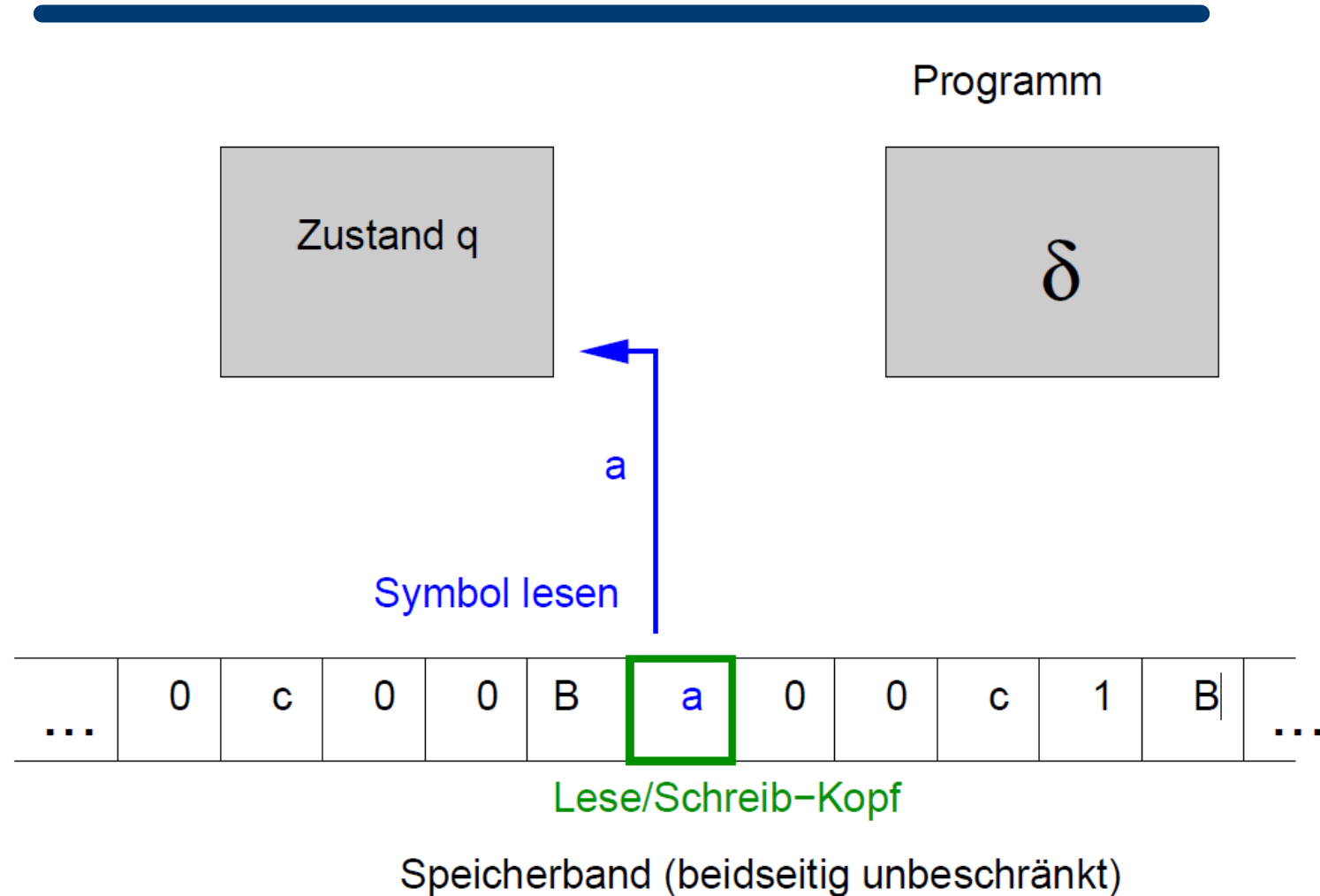
Programm



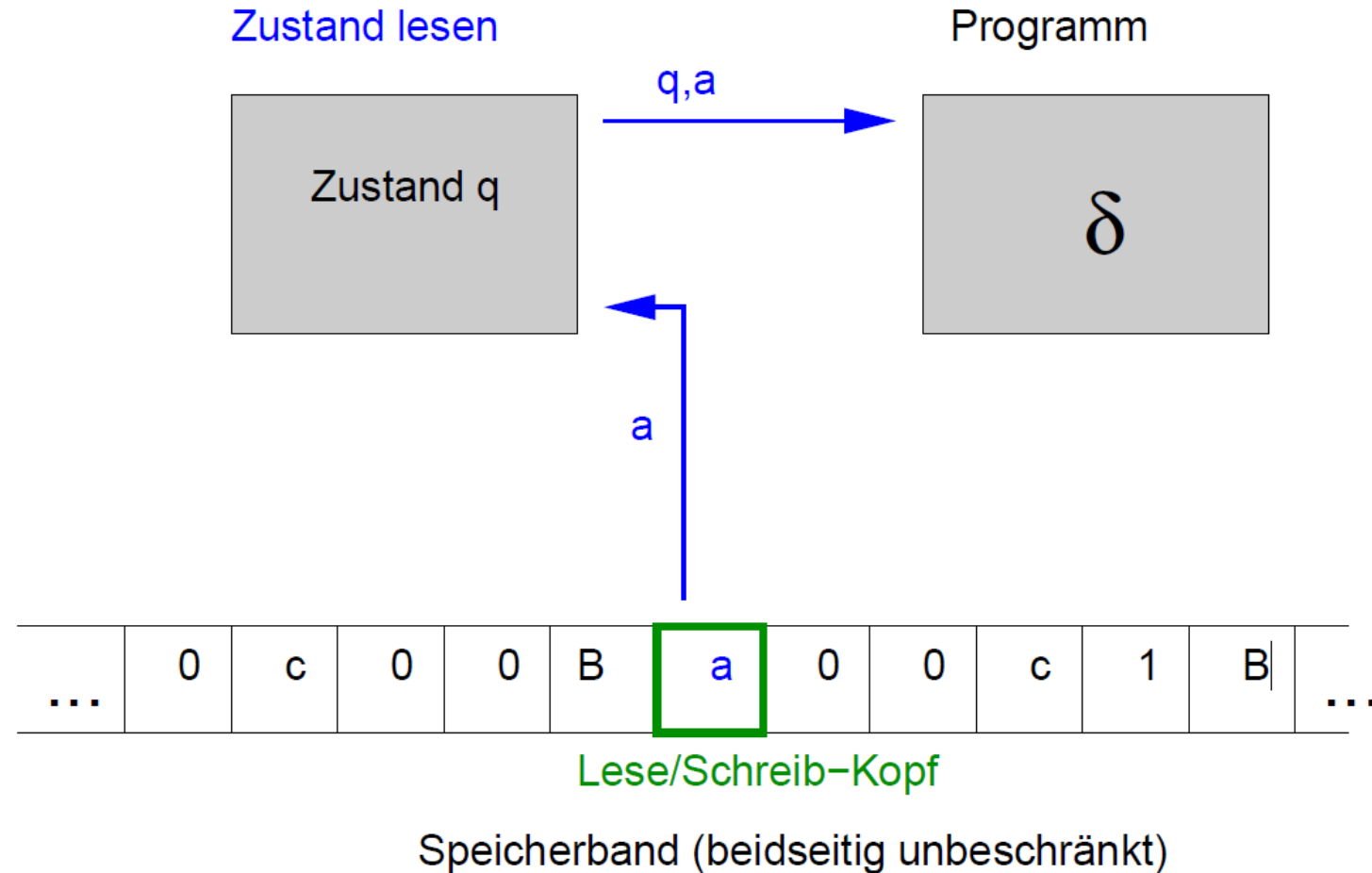
Lese/Schreib-Kopf

Speicherband (beidseitig unbeschränkt)

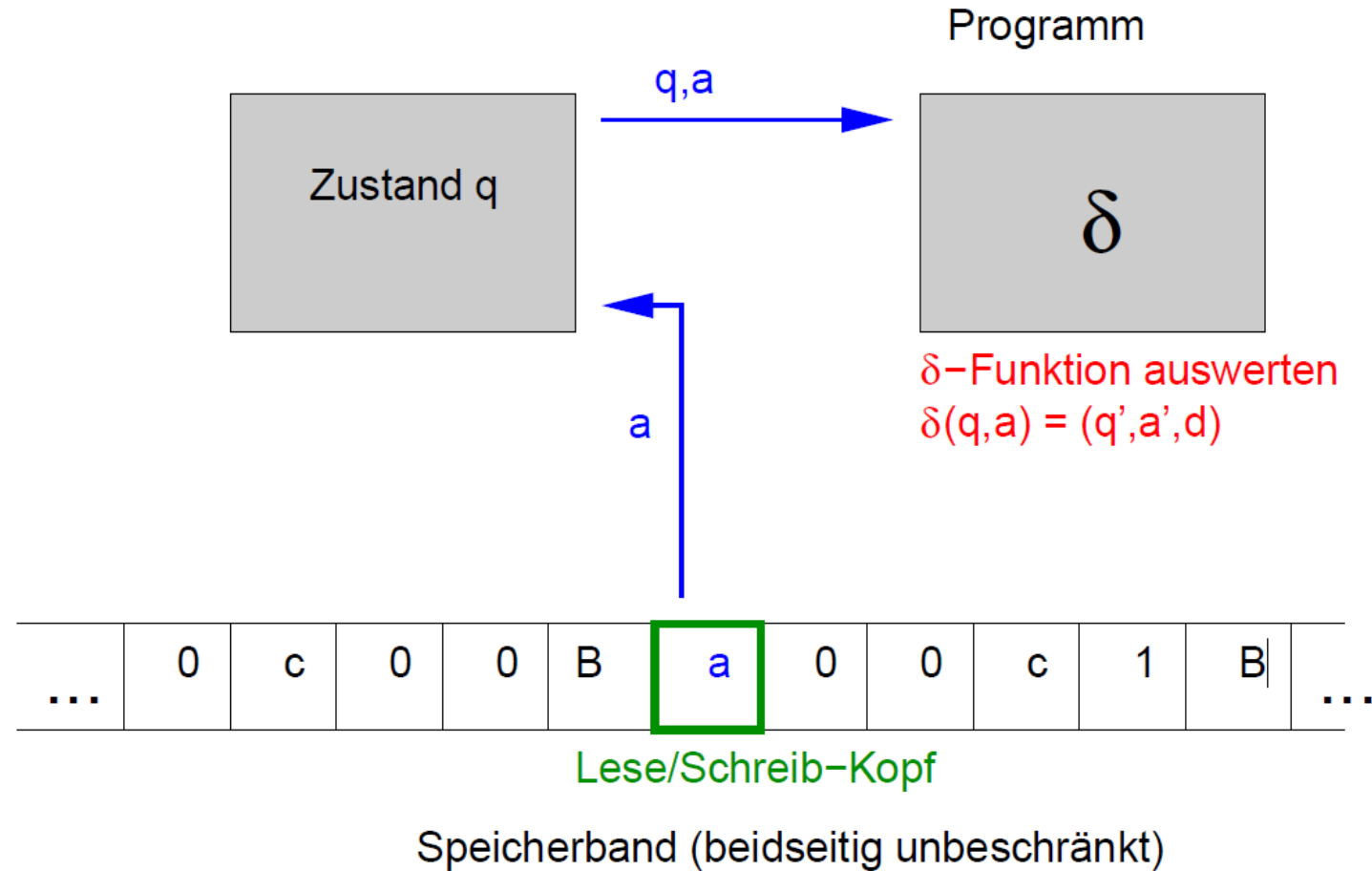
Deterministische Turingmaschinen (TM bzw. DTM)



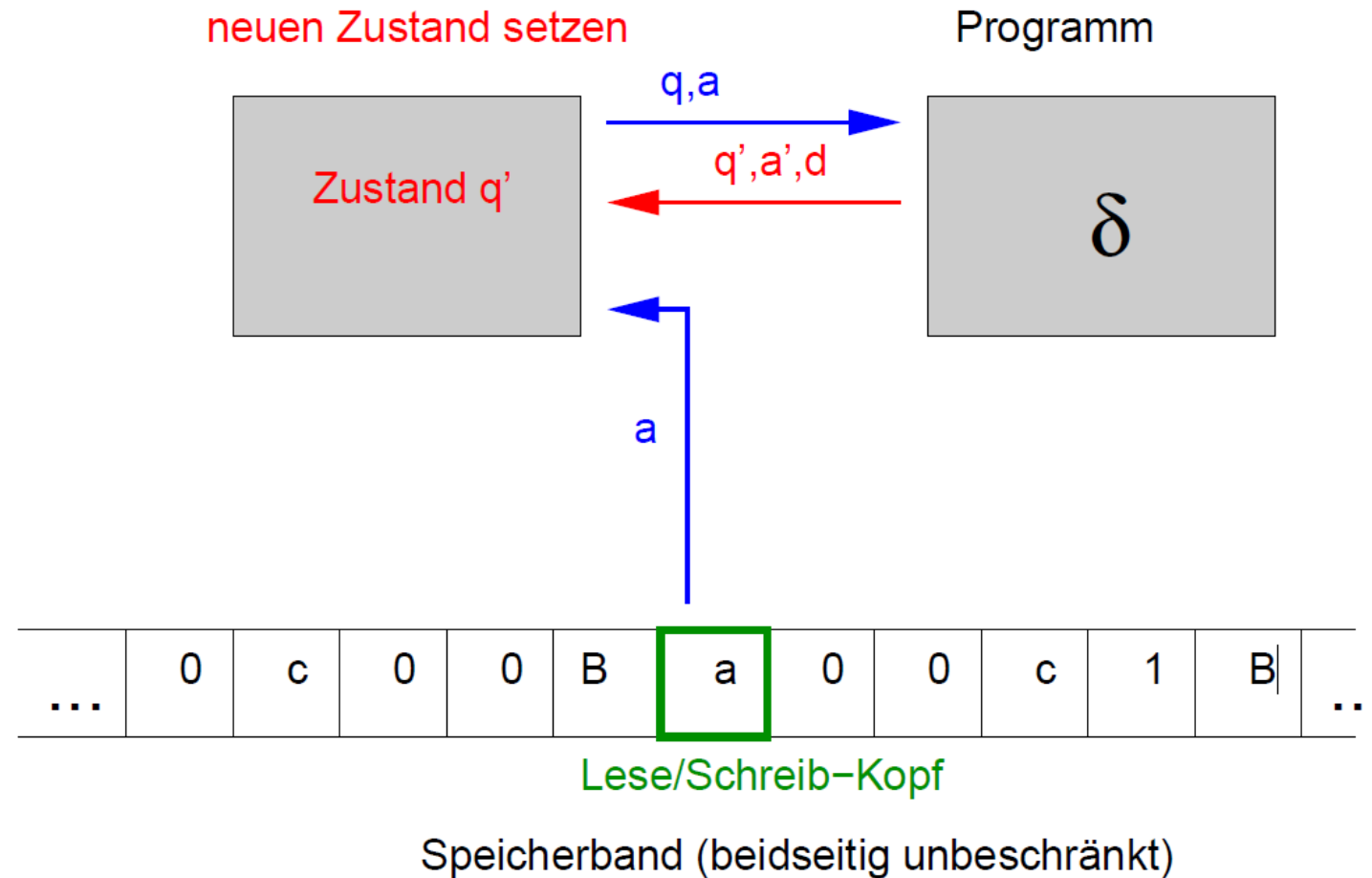
Deterministische Turingmaschinen (TM bzw. DTM)



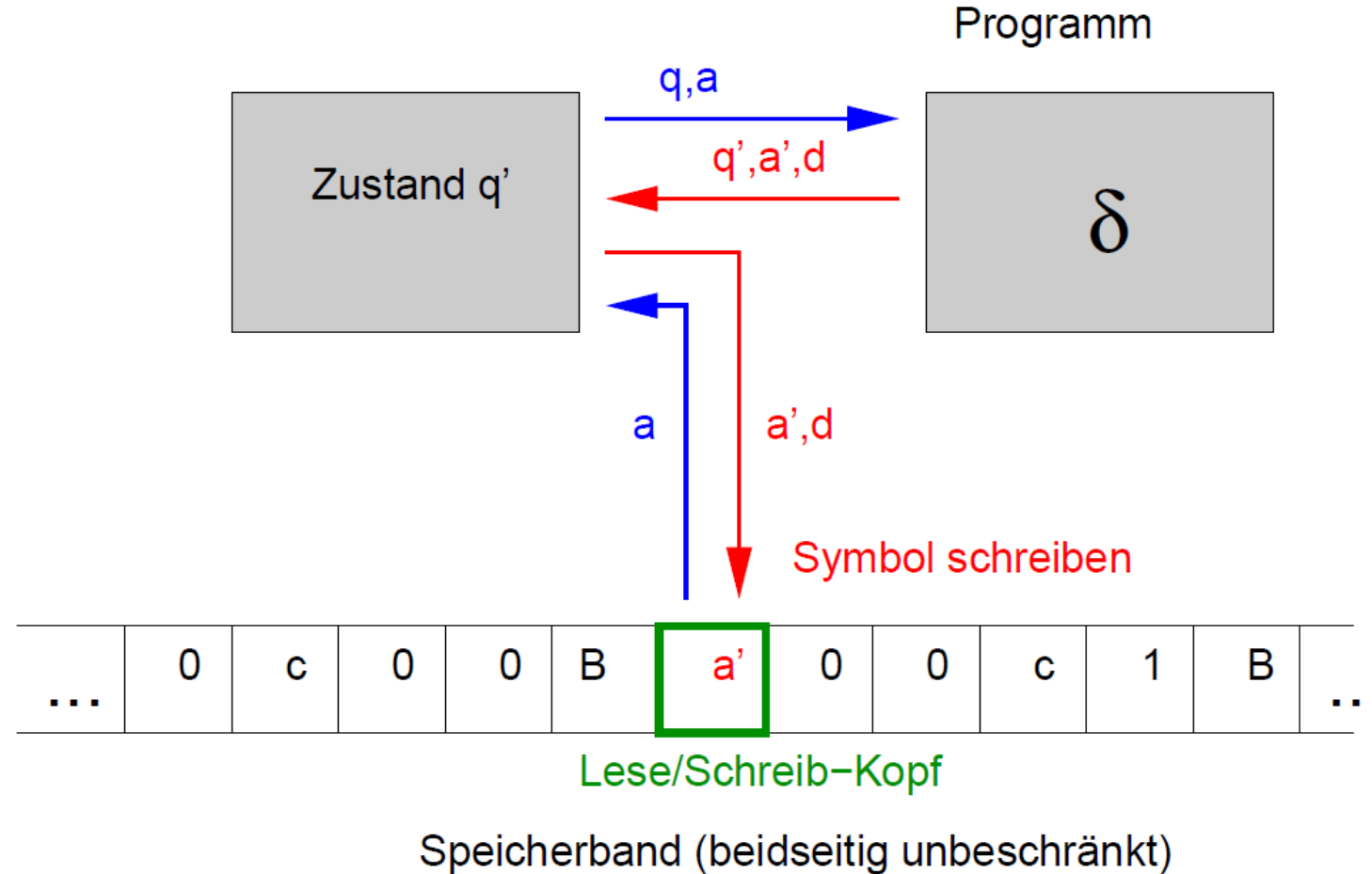
Deterministische Turingmaschinen (TM bzw. DTM)



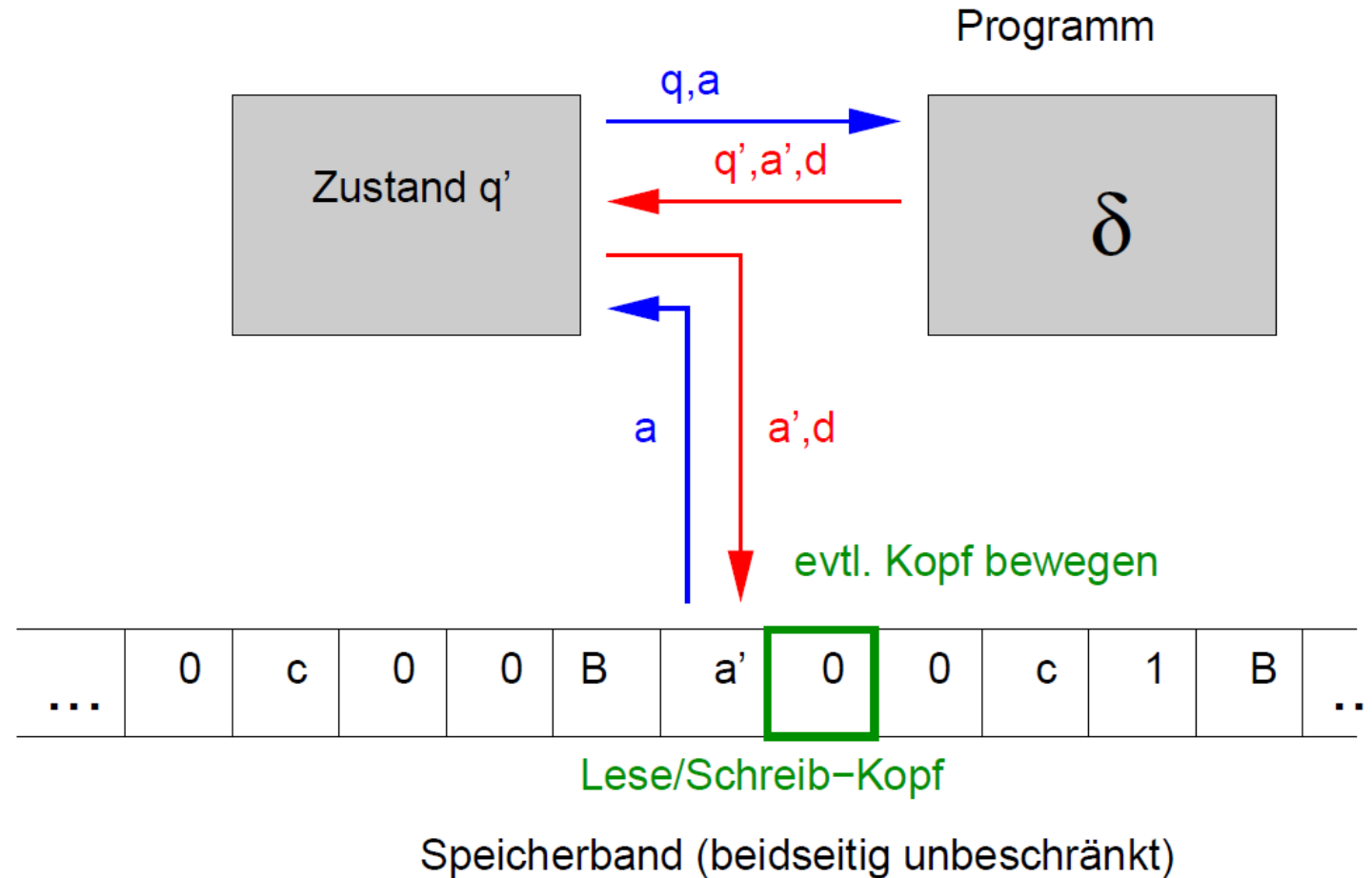
Deterministische Turingmaschinen (TM bzw. DTM)



Deterministische Turingmaschinen (TM bzw. DTM)



Deterministische Turingmaschinen (TM bzw. DTM)



Komponenten einer Turingmaschine

- Q die endliche Zustandsmenge
- Σ das endliche Eingabealphabet
- Γ das endliche Bandalphabet mit $\Sigma \subset \Gamma$
- $B \in \Gamma - \Sigma$ das Leerzeichen (Blank)
- $q_0 \in Q$ der Anfangszustand
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{R, L, N\}$ die Zustandsüberföhrungsfunktion
- $F \subseteq Q$ die Menge der akzeptierenden Endzustände
- Eine **Turingmaschine** ist definiert durch das 7-Tupel $(Q, \Sigma, \Gamma, B, q_0, \delta, F)$

Funktionsweise einer TM

- Ausgangssituation
 - auf dem Band steht B^*wB^* , wobei $w \in \Sigma^*$ die Eingabe ist
 - der initiale Zustand ist q_0
 - der Kopf steht über dem ersten Symbol von w
- Durchführung eines Rechenschrittes
 - $a \in \Gamma$ bezeichne das gelesene Symbol
 - $q \in Q$ bezeichne den aktuellen Zustand
 - es sei $\delta(q, a) = (q', a', d)$, für $q' \in Q, a \in \Gamma, d \in \{L, R, N\}$
 - dann wird der Zustand auf q' gesetzt
 - an der Kopfposition wird das Symbol a' geschrieben
 - der Kopf bewegt sich
 - um eine Position nach rechts, falls $d = R$
 - um eine Position nach links, falls $d = L$
 - nicht, falls $d = N$

Funktionsweise einer TM

- Ende der Rechnung
 - die TM stoppt, wenn sie sich in einem Zustand $q \in Q$ befindet, $a \in \Gamma$ liest und es gilt $\delta(q, a) = (q, a, N)$
 - das Ausgabewort w' kann dann vom Band abgelesen werden:
 - w' beginnt an der Kopfposition und endet unmittelbar vor dem ersten Leerzeichen rechts von dieser Position
 - Spezialfall: bei Entscheidungsproblemen ist die Antwort im Zustand kodiert:
 - die TM akzeptiert das Eingabewort, wenn sie in einem Zustand $q \in F$ terminiert
 - die TM verwirft das Eingabewort, wenn sie in einem Zustand $q \notin F$ terminiert
- Bemerkungen:
 - Laufzeit = Anzahl der Schritte
 - beachte, möglicherweise terminiert die Rechnung nicht

Funktionsweise einer TM am Beispiel

- Die Sprache $L = \{w1 | w \in \{0,1\}^*\}$ enthält alle Binärwörter, die mit einer 1 enden.
- Die Sprache L wird entschieden (wir sagen auch erkannt) durch die TM $(Q, \Sigma, \Gamma, B, q_0, \delta, F)$ mit
 - $Q = \{q_0, q_1, q_2\}$
 - $\Sigma = \{0,1\}$
 - $\Gamma = \{0,1, B\}$
 - $F = \{q_2\}$
 - δ gemäß Tabelle
- Einträge, in denen die Maschine stoppt, sind der besseren Lesbarkeit halber mit "–" gekennzeichnet.

δ	0	1	B
q_0	$(q_0, 0, R)$	$(q_1, 1, R)$	–
q_1	$(q_0, 0, R)$	$(q_1, 1, R)$	(q_2, B, N)
q_2	–	–	–